



WYDZIAŁ ELEKTRONIKI,
TELEKOMUNIKACJI
I INFORMATYKI

Dokumentacja projektu grupowego na przedmiot ZSD

Dokumentacja techniczna projektu

Wydział Elektroniki, Telekomunikacji i Informatyki

Politechnika Gdańska

Nazwa i akronim projektu: CSAimBot, CSBot	Zleceniodawca: dr hab. inż. Michał Czubenko	
Numer zlecenia: -	Kierownik projektu: Nazar Tomasz	Opiekun projektu: mgr. inż. Jan Glinko

Nazwa dokumentu/akronim: Dokumentacja techniczna projektu – DTP	Nr wersji: 1.02
Odpowiedzialny za dokument: Pietrzak Filip Nazar Tomasz Puczyński Karol Antoni Sulkowski	Data pierwszego sporządzenia: 7.04.2026
	Data ostatniej aktualizacji: 17.06.2026
	Studia I stopnia, inżynierskie
	<i>Semestr realizacji Projektu grupowego: 2</i>

Historia zmian

Wersja	Opis modyfikacji	Rozdział / strona	Autor modyfikacji	Data
1.00	Wstępna wersja	całość	Pietrzak Filip	7.04.2026
1.01	Wstępne opisanie aplikacji	2.3	Puczyński Karol	19.05.2026
1.02	Aktualizacja po projekcie	całość	Nazar Tomasz, Sulkowski Antoni	17.06.2026

Spis treści

1	Wprowadzenie - o dokumencie	2
1.1	Cel dokumentu	2
1.2	Zakres dokumentu	2
1.3	Odbiorcy	2
1.4	Terminologia	2
2	Założenia i cele projektu	2
2.1	Cel projektu	2
2.2	Zakres funkcjonalny	2
3	Szczegółowe założenia	2
3.2.1	Hardware'owe	3
3.2.2	Software'owe	3
3.2.3	Związane z grą	4
	Stack Technologiczny	4
3.2	Software	4
3.3	Hardware	4
3.4	Milestone 1: Wstępny model i projekt sprzętowy	5
3.4.1	M1 - Software (Wykrywanie w YOLO):	5
3.4.2	M1 - Hardware (Projekt CAD):	5
3.5	Milestone 2: Konstrukcja i sterowanie ręczne	5
3.5.1	M2 - Software (Aplikacja i integracja wejścia):	5
3.5.2	M2 - Hardware (Fizyczny robot):	5
3.6	Milestone 3: Pełna integracja (Auto-Aim i kalibracja)	6
3.6.1	M3 - Software + Hardware:	6
4	Dokumentacja techniczna projektu	6
4.1	YOLO oraz Dataset	6
4.1.1	Rozmiar datasetu	6
4.1.2	Struktura zdjęć	6
4.1.3	Opis zdjęć - YOLO	6
4.2	Robot – fizyczny i symulacja	6
4.2.1	Materiały i wykaz elementów	6
4.2.2	CoreXY – technologia	7
4.2.3	Symulacja - technologia	7
4.2.4	Screenshot z symulatora Gazebo	7
4.2.5	Modele 3D - technologia	8
4.3	Aplikacja i komunikacja z ESP32	8
4.3.1	Architektura Oprogramowania Aplikacji Głównej	8
4.3.2	Struktura klas i modułów	9
4.3.3	Architektura interfejsu graficznego oraz inicjalizacja systemu	10
5	Napotkane problemy	11
5.1	Milestone I	11
5.2	Milestone II	11
5.3	Milestone III	11
6	Perspektywy rozwinięcia projektu	11
7	Załączniki	12

1 Wprowadzenie - o dokumencie

1.1 Cel dokumentu

{Nie zmieniać}

Celem dokumentu jest udokumentowanie informacji dotyczących produktu, jego cech funkcjonalnych, parametrów technicznych, schematów blokowych, oprogramowania, wyników działania, zdjęć produktu, pomiarów, testów oraz innych elementów wymaganych przez opiekuna i klienta.

1.2 Zakres dokumentu

{Określenie, co wchodzi w zakres dokumentu, a co nie wchodzi, ew. wskazanie na dokumenty powiązane}

Dokument zawiera szczegóły techniczne projektu takie jak: rozmiary, technologie wykonania oraz szczegóły kodu. Dokument nie zawiera szczegółów na temat tego w jakiej kolejności powstały poszczególne części projektu oraz kto jest za nie odpowiedzialny – te opisy są raportowane na wykładach z zajęć z przedmiotu Zintegrowane systemy decyzyjne oraz na repozytorium na githubie

1.3 Odbiorcy

{Określenie adresatów dokumentu, może być to typ odbiorcy; tu: zleceńbiorca (Katedra), członkowie zespołu projektowego oraz wymienione z nazwiska osoby, do których dokument ma dotrzeć}

Odbiorcami projektu są dr hab. inż. Michał Czubenko oraz mgr inż. Jan Glinko.

1.4 Terminologia

{Wyjaśnienie używanych w dokumencie pojęć i skrótów, oznaczenia używane wewnątrz dokumentu np. oznaczenia wymagań}

Kierownik – kierownik zespołu - Tomasz Nazar

Zastępca kierownika – Filip Pietrzak

Dokument – Niniejszy dokument DTP

Zdjęcia z datasetu - screenshoty z gry Counter Strike: Global Offensive

CS:GO – oryginalna wersja gry Counter Strike: Global Offensive

eDPI - wartość uzyskana po przemnożeniu sensivity – ustawionej czułości w CS:GO – z DPI – ustawionej na komputerze czułości w myszce.

2 Założenia i cele projektu

2.1 Cel projektu

Celem projektu było stworzenie wpelni funkcjonalnego robota kartezjańskiego zdolnego do samodzielnego celowania w grze CS:GO. Robot miał za zadanie samemu chwycić myszkę i namierzyć wroga na podstawie koordynatów wysyłanych przez system wizyjny.

2.2 Zakres funkcjonalny

- Czytanie ekranu
- Oznaczanie przeciwników
- Obliczanie drogi robota
- Ruch myszką za pomocą robota kartezjańskiego
- Strzał za pomocą przycisku
- Rozglądanie się w przypadku braku wykrycia przeciwników
- Współpraca z graczem
- Symulacja robota przez ROS i Gazebo

3 Szczegółowe założenia

3.1.1 Hardware'owe

- Projekt będzie działać w temperaturze pokojowej w zamkniętym pomieszczeniu na jednolitej poziomej płaskiej powierzchni umożliwiającej poprawne zamontowanie robota (np. biurko) i umożliwiającej poprawne użycie klawiatury.
- Robot będzie rozmiarów umożliwiających swobodne sterowanie myszką i wygodnie sterowanie klawiaturą przez użytkownika na jednym stanowisku.
- Mysz, będzie się poruszać na materiałowej podkładce do myszy.
- Komputer i robot, będą pobierały zasilanie z sieci 230V. Dalsze etapy zasilania (np. zasilacz komputerowy), realizowane w zależności od zastosowanego sprzętu.
- Wszystkie komponenty muszą być sprawne, aby projekt działał poprawnie.
- Pozycja silników krokowych będzie odczytywana z enkoderów podłączonych w odpowiedni sposób do silników.
- Mysz użyta do projektu powinna być bezprzewodowa lub powinna mieć kabel o długości zapewniającej swobodną pracę robota.

- Sterowanie robotem będzie odbywać się WYŁĄCZNIE za pomocą klawiatury i/lub myszki oraz dedykowanych przycisków przy robocie
- Mysz musi posiadać przynajmniej 2 przyciski - lewy i prawy przycisk, reszta przycisków będzie ignorowana.
- Mysz musi korzystać z laserowego lub optycznego sensora. Nie może być to mysz kulkowa. Mysz nie może mieć znacznie wystających niestandardowych elementów.
- Przy zmianie myszki/wartości DPI na inną wykonywana będzie półautomatyczna procedura kalibracji. Robot będzie sprawdzał o ile musi się poruszyć, żeby przejść między punktami referencyjnymi na specjalnej mapie.
- Klawiatura będzie obsługiwana przez jednego użytkownika, steruje poruszaniem się postacią klawiszami WSAD
- Projekt nie będzie obsługiwać komend głosowych ani wizji maszynowej do poleceń.
- Chwytnik będzie zamykany serwomechanizmem
- Mysz będzie trzymana siłą tarcia
- Mysz pod chwytnikiem robota umieszcza użytkownik w miejscu do tego przeznaczonym (centrum podkładki).
- Nastawy robota będą mogły być zmienione w przypadku poluzowania się części lub wykrycia innych nieprawidłowości.
- Gracz ma do dyspozycji przycisk zatrzymania robota.
- Komunikacja PC-Mikrokontroler poprzez UART (USB) z prędkością 115200 baud
- Przyciski myszy będą naciskane solenoidami lub szybkimi serwomechanizmami.
- Mysz musi mieć klawisze (lewy i prawy) równoległe do powierzchni na której stoi robot. Nie obsługiwane będą myszy typu Logitech MX Vertical itp.
- Akceleracja wskaźnika myszy jest wyłączona.
- Elementy robota oprócz silników, szyn, sterowników, przewodnic, śrub, zębatek, pasów itp. będą wydrukowane 3d

3.1.2 Software'owe

- Program komputerowy będzie działać na systemie Windows 11 Pro build $\geq 25H2$ lub równoważnym systemie Windows. Program nie będzie działać na Linuxie ani na MacOS.
- Stworzony zostanie własny dataset do wykrywania przeciwników oraz rodzaju trzymanej broni
- Program będzie działać na komputerze PC o odpowiedniej mocy obliczeniowej (min. karta graficzna RTX 3060) z monitorem - nie na laptopie. Na laptopie może nie zostać spełniony warunek czasu reakcji.
- Program będzie wykrywał modele przeciwników w grze. (Będzie odróżniał modele CT od T)
- Program będzie współpracować z CS:GO w wersji umożliwiającej dodanie botów lub graczy do gry.
- Program będzie komunikował się z robotem za pomocą połączenia przewodowego.
- Program będzie przystosowany do działania lokalnie na komputerze na którym zostanie zainstalowany.
- Ewentualny interfejs użytkownika będzie zrealizowany w języku angielskim.
- Jeśli gracz trzyma pistolet lub karabin snajperski to kompensacja odrzutu będzie wyłączona. W przeciwnym wypadku program będzie korygował odrzut konkretnych broni (AK47/M4A1).
- Podczas celowania z karabinu snajperskiego wykrywani zostaną tylko wrogowie znajdujący się w polu widzenia lunety. Włączeniem / Wyłączeniem przybliżenia steruje użytkownik z klawiatury.
- Użytkownik będzie używał praworęcznego modelu broni.
- Robot będzie strzelał pojedynczymi strzałami lub seriami w zależności od ustawienia gracza. Odrzut będzie kompensowany jedynie przy strzelaniu seriami.

- Program będzie działał z grą ustawioną na minimum 60 FPS.
- Okno wykrywania będzie miało rozmiar 1280x736
- Robot będzie miał 100% celności w cele statyczne.
- Robot będzie miał 60% celności w cele poruszające się.
- Trenowanie YOLO - na komputerze
- Rozdzielczość 1080p
- ROS obsługiwany za pomocą Dockera i WSL2 na Windows

3.1.3 Związane z grą

- Program będzie działał tylko podczas rozpoczętej rundy.
- Program będzie działał na dowolnej oficjalnej mapie CS:GO
- Program będzie działał z włączoną opcją natychmiastowego usuwania ciał po zabójstwie w grze lokalnej
- W przypadku wykrycia kilku celów, program wybiera cel najbliższej aktualnej pozycji celownika
- Czas reakcji przy eDPI dobranym tak, aby robot nie musiał podnosić myszy będzie wynosił maksymalnie tyle ile czas reakcji dobrego gracza (<200ms) Czasem reakcji nazywamy czas po którym robot zaczyna kierować mysz w stronę przeciwnika od jego pojawienia się na ekranie.
- Viewmodel będzie ustawiony tak aby było go widać w obszarze przechwytywania

Stack Technologiczny

3.2 Software

- PyCharm >=2025.3.2
- Visual Studio Code >=1.108
- Arduino IDE >=2.3.5
- Python >=3.12
- YOLO26
- Docker + ROS image
- GitHub + Git
- CS:GO - oficjalna wersja legacy
- Autodesk Fusion
- Gazebo
- Bambu Studio

3.3 Hardware

- Esp32
- 2x silniki NEMA 23
- 3x sterowniki silników krokowych DM556
- Prowadnice
- Szyny
- Śruba kulowa/trapezowa
- Silnik NEMA 17
- Solenoid
- Logic level converter 3.3V -> 5V
- Zasilacz 36V 10A
- Filament TPU
- Filament PLA/PETG
- Kable
- Nvidia RTX 3060

3.4 Milestone 1: Wstępny model i projekt sprzętowy

3.4.1 M1 - Software (Wykrywanie w YOLO):

- **Cel:** Przetwarzanie obrazu i lokalizacja przeciwników na ekranie, komunikacja z mikrokontrolerem.
- **Mierzalny wskaźnik:** Średnia precyzja, szybkość przetwarzania (FPS) oraz rozmiar datasetu.
- **Dataset / Sposób pomiaru:** Ewaluacja na zbiorze testowym złożonym ze screenów z gry na różnych mapach z użyciem wbudowanych skryptów YOLO. Prędkość mierzona podczas gry lokalnej.
- **Warunek przejścia dalej:** Skuteczność YOLO > 80% dla wykrywania modeli CT/T, stałe utrzymanie min. 60 FPS w czasie inferencji na karcie RTX 3060, dataset o rozmiarze min 5000 zdjęć.

3.4.2 M1 - Hardware (Projekt CAD):

- **Cel:** Zaprojektowanie robota kartezjańskiego zdolnego do przemieszczania i podnoszenia myszki oraz przygotowanie kosztorysu
- **Mierzalny wskaźnik:** Osiągalna przestrzeń robocza oraz teoretyczny luz mechaniczny.
- **Sposób pomiaru:** Symulacja kinematyki i pomiary w programie typu CAD (np. Autodesk Fusion).
- **Warunek przejścia dalej:** Projekt pozwala na pełne pokrycie wirtualnej podkładki o wymiarach min. 30x30 cm, podniesienie uchwytu z myszką w osi Z na wysokość gwarantującą brak odczytu sensora oraz zakupienie części.

3.5 Milestone 2: Konstrukcja i sterowanie ręczne

3.5.1 M2 - Software (Aplikacja i integracja wejścia):

- **Cel:** Aplikacja PC czytająca klawiaturę i wysyłająca komendy do mikrokontrolera, symulacja w Gazebo, kompensacja odrzutu broni.
- **Mierzalny wskaźnik:** Opóźnienie przesyłu danych (Latency) na linii PC -> Mikrokontroler. Rozmiar datasetu oraz skuteczność Yolo.
- **Sposób pomiaru:** Analiza logów z timestampami wysłania paczki danych oraz potwierdzenia jej odbioru przez UART (115200 baud).
- **Warunek przejścia dalej:** Skuteczność YOLO > 95%, opóźnienie wysłania pakietu poniżej 5 ms i ~0% zgubionych ramek na dystansie 10 tysięcy prób, dataset o rozmiarze min 10000 zdjęć.

3.5.2 M2 - Hardware (Fizyczny robot):

- **Cel:** Budowa fizycznego robota i weryfikacja poprawności sterowania silnikami z poziomu aplikacji.
- **Mierzalny wskaźnik:** Czas reakcji mechaniki na komendę.
- **Sposób pomiaru:** Nagranie w zwolnionym tempie od momentu wciśnięcia przycisku klawiatury do rozpoczęcia ruchu silnika krokowego/serwa.
- **Warunek przejścia dalej:** Wydrukowane elementy 3D, reakcja silników i fizyczne poruszenie myszką. Czas reakcji < 200ms.

3.6 Milestone 3: Pełna integracja (Auto-Aim i kalibracja)

3.6.1 M3 - Software + Hardware:

- **Cel:** Integracja całego systemu: YOLO podaje koordynaty, algorytm przelicza je na kroki, a robot fizycznie namierza i strzela.
- **Mierzalny wskaźnik:** Całkowity czas reakcji układu, celność strzału oraz skuteczność wykrywania broni po stronie gracza.
- **Sposób pomiaru:** Testy na lokalnym serwerze treningowym (np. Aim Botz). Mierzenie poprzez analizę nagrania z rozgrywki.
- **Warunki przejścia dalej:**

- Całkowity czas reakcji układu (Wykrycie -> Ruch) wynosi < 200 ms.
- Dokładność namierzania stojącego celu 100%.
- Półautomatyczna procedura kalibracji eDPI/myszki działa poprawnie.
- System poprawnie włącza/wyłącza kompensację odrzutu na podstawie trzymanej broni (AK47/M4A1 vs Pistolet/Snajperka).

4 Dokumentacja techniczna projektu

{O zakresie dokumentacji decyduje opiekun, tu należy zacząć opis rozwiązania technicznego wg zaleceń opiekuna, w układzie redakcyjnym najlepiej oddającym charakter projektu – kolejne rozdziały, podrozdziały, punkty}

4.1 YOLO oraz Dataset

4.1.1 Rozmiar datasetu

Zdjęć do datasetu jest 10461.

4.1.2 Struktura zdjęć

Zdjęcia wykonywane są przy pomocy skryptu pythonowego w wielkości 1280x736 [px] dla Full HD oraz 1728x992 [px] dla 2K (wielokrotności 32 px) oraz skalowane do 1280x736px.

4.1.3 Opis zdjęć - YOLO

Opis zdjęć wykonywany jest w Roboflow, na dzień pierwszej wersji dokumentu (7.04.2026) zdjęć opisanych jest ponad 6 tysięcy.

4.2 Robot – fizyczny i symulacja

4.2.1 Materiały i wykaz elementów

1. Mechanika
 - Szyna liniowa MGN12 z wózkiem (40 cm) – 2 szt.
 - Szyna liniowa MGN12 z wózkiem (50 cm) – 1 szt.
 - Profil aluminiowy T-slot 2020 (40 cm) – 4 szt.
 - Narożniki aluminiowe (wersja 20) – 10 szt.
 - Pas zębaty GT2 (5 m, 10 mm szerokości) – 1 szt.
 - Rolki zębate GT2 – 6 szt.
 - Rolki gładkie GT2 – 2 szt.
 - Zębátki napędowe na wał silnika 8 mm – 2 szt.
 - Śruba trapezowa osi Z (skok 2 mm, 10 cm) – 1 szt.
 - Sprzęgło 8x5 mm – 1 szt.
 - Pręt stalowy 3 mm (długość 330 mm) – 2 szt.
 - Filament (TPU, 1,75mm, 1 kg) – 1 szt.
2. Silniki i sterowniki
 - Silnik krokowy Nema 23 (dwustronny wał 8 mm) – 2 szt.
 - Silnik krokowy Nema 17 "Pancake" – 1 szt.
 - Sterownik cyfrowy DM556 – 3 szt.
 - Enkoder obrotowy E6B2-CWZ6C (600 P/R) – 2 szt.
3. Chwyty i klikanie
 - Serwo MG996R (180 stopni) – 1 szt.
 - Zębátka serwa 25T (Moduł 0.5) – 1 szt.
 - Listwa zębata (Moduł 0.5, 5x5x300mm) – 1 szt.
 - Solenoidy 12V – 2 szt.
4. Elektronika i sterowanie
 - Mikrokontroler ESP32 – 1 szt.
 - Konwerter poziomów logicznych TXS0108E (8-kanalowy) – 1 szt.
 - Przetwornica LM2596 ADJ (Regulowana) – 2 szt.
5. Zasilanie

- Zasilacz (36V, 10A, 360W) – 1 szt.
6. Dodatkowe części drukowane
- Chwytnik Fin Ray – 2 szt.
 - Podstawy do silnika – 2 szt.
 - Elementy wydrukowane (na Githubie)

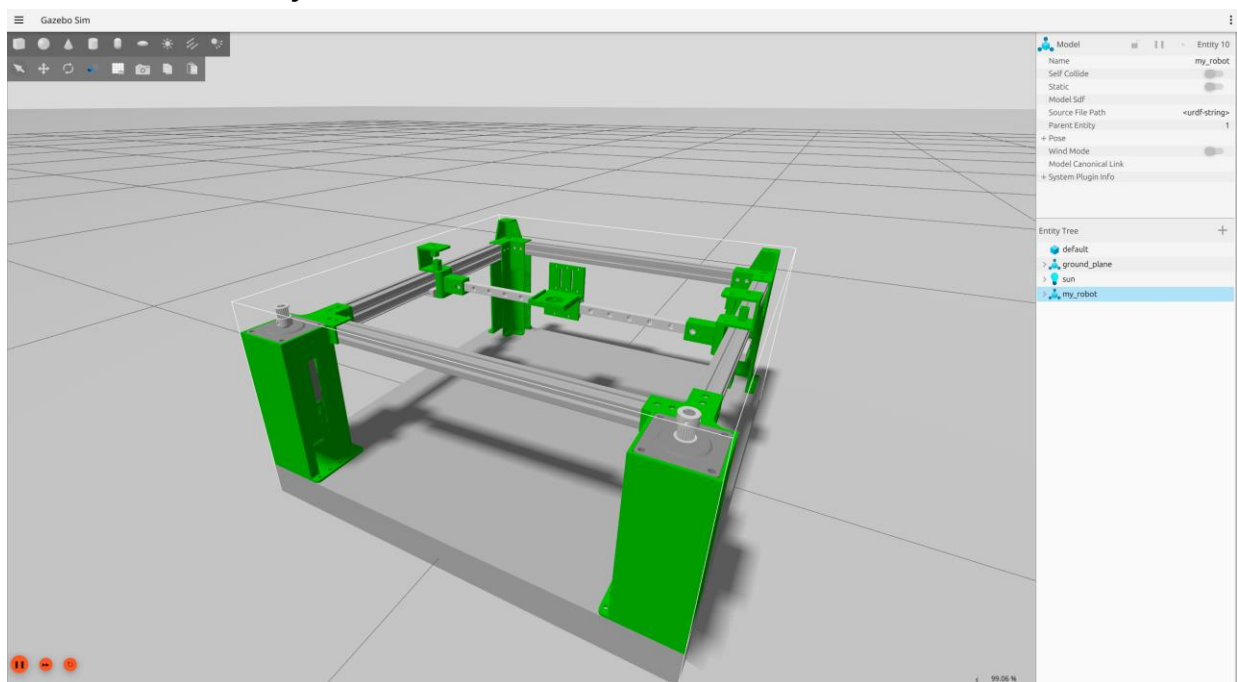
4.2.2 CoreXY – technologia

Wybór kinematyki CoreXY w projekcie CSAimBot jest podyktowany wymogiem uzyskania czasu reakcji poniżej 200 ms. Architektura ta opiera się na stacjonarnym montażu silników napędowych na ramie głównej, podczas gdy ruch wózka realizowany jest za pośrednictwem układu pasów zębatych. Rozwiązanie to drastycznie zmniejsza masę ruchomą i inercję całego systemu.

4.2.3 Symulacja - technologia

Symulacja została zrealizowana przy użyciu środowiska ROS. Wirtualny model robota oraz fizyka otoczenia symulowane są w programie Gazebo, z kolei narzędzie RViz odpowiada za graficzną reprezentację danych z systemu sterowania.

4.2.4 Screenshot z symulatora Gazebo



4.2.5 Modele 3D - technologia

Modele 3D, projektowane w oprogramowaniu Autodesk Fusion, służą zarówno do stworzenia wirtualnej symulacji w środowisku Gazebo, jak i do fizycznej budowy robota kartezjańskiego z wykorzystaniem technologii druku 3D.

4.3 Aplikacja i komunikacja z ESP32

4.3.1 Architektura Oprogramowania Aplikacji Głównej

Aplikacja sterująca robotem CSAimBot została zaprojektowana w oparciu o architekturę wieloprotokółową. Rozdzielenie tych zadań na osobne procesy pozwala na ich równoległe wykonywanie i zapobiega blokowaniu interfejsu użytkownika podczas obciążających operacji wejścia/wyjścia lub obliczeń na GPU.

System składa się z jednego procesu głównego oraz dedykowanych procesów pomocniczych:

- **Proces Główny (*app.py*):** Punkt wejścia aplikacji. Inicjuje wszystkie procesy potomne, zarządza kanałami komunikacyjnymi. Proces ten centralizuje przepływ informacji – odbiera dane takie jak status z ESP32, czy potrzebny dystans do pokonania przez robota do trafienia w przeciwnika od procesów potomnych i dystrybuje do nich odpowiednie komendy (np. zmianę priorytetu celu).

- **Vision Worker (*aimbot.py*):** Niezależny proces odpowiedzialny za cały potok wizyjny. Realizuje asynchroniczne przechwytywanie klatek obrazu z ekranu, transfer danych bezpośrednio do pamięci VRAM oraz inferencję modelu do detekcji przeciwników. Wynikiem działania tego procesu jest ciągły strumień obliczonych przesunięć w osiach X i Y względem środka ekranu do wroga, przesyłany z określoną częstotliwością do procesu głównego.
- **Comms Worker (*comms.py*):** Proces odpowiadający za dwukierunkową komunikację sprzętową. Utrzymuje i monitoruje połączenie szeregowo z ESP32, asynchronicznie przesyłając ramki sterujące (zawierające m.in. wyliczone przesunięcia, stany przycisków myszy oraz prędkość) i odbierając statusy urządzenia.
- **Simulation Worker (*sim_worker.py*):** Proces odseparowujący logikę sterowania środowiska symulacyjnego od warstwy fizycznej robota. Zarządza komunikacją ze środowiskiem ROS2/Gazebo oraz symulowaną kinematyką układu.
- **Mouse Blocker Worker (*mouse_blocker_worker.py*):** Proces odpowiedzialny za programowe filtrowanie wejść z myszy fizycznej. Jego zadaniem jest zablokowanie dodatkowych przycisków i funkcji (takich jak przycisk środkowy, rolka), przepuszczając wyłącznie ruch oraz kliknięcia lewym i prawym przyciskiem myszy. Mechanizm ten uniemożliwia rejestrowanie przypadkowych akcji podczas fizycznego zaciskania chwytaka robota na obudowie myszy, co mogłoby zdestabilizować działanie programu. Proces ten aktywowany jest równolegle z procesem *Vision Worker*.

4.3.2 Struktura klas i modułów

W celu zapewnienia pełnej modułowości oprogramowania oraz separacji zadań, logika aplikacyjna podsystemu sterowania, komunikacji i detekcji została podzielona na niezależne moduły. Każdy z modułów odpowiada za realizację ściśle zdefiniowanego zadania i wykonuje się w odrębnym procesie. Poniżej przedstawiono szczegółowy opis struktury klas oraz metod wchodzących w skład struktury projektu.

Moduł wizyjny i inferencji (*aimbot.py*) Moduł ten stanowi rdzeń potoku przetwarzania obrazu. Odpowiada za realizację inferencji modelu YOLO w czasie rzeczywistym oraz wyznaczanie błędu między pozycją aktualną a zadaną.

- **Klasa AimBot:** Główna klasa zarządzająca potokiem wizyjnym. Odpowiada za konfigurację regionu przechwytywania, przygotowanie modelu YOLO oraz statyczną alokację zmiennych w pamięci GPU.
 - *prepare_camera()*: Definiuje statyczny obszar przechwytywania ekranu (FOV o rozdzielczości 1280x736 pikseli) dopasowany do środka rozdzielczości monitora głównego.
 - *prepare_model(model_path)*: Odpowiada za inicjalizację instancji sieci YOLO oraz wykonanie fazy rozgrzewkowej przy użyciu pustego tensora w pamięci CUDA w celu eliminacji opóźnień przy pierwszej klatce.
 - *allocate_variables()*: Dokonuje alokacji obiektów i tensorów w pamięci VRAM karty graficznej.
 - *capture_and_preprocess_frame()*: Pobiera surowy tensor z providera kamery, normalizuje wartości pikseli do zakresu [0.0, 1.0] i przepisuje dane bezpośrednio do tensora wejściowego modelu.
 - *calculate_best_target_position(boxes_data_tensor)*: Realizuje algorytm wyboru celu. Nadaje najwyższy priorytet klasom powiązanym z głową przeciwnika, a w przypadku ich braku przełącza się na klasy korpusu. W obrębie wybranej grupy priorytetowej selekcionuje obiekt położony najbliżej środka ekranu i wylicza przesunięcie w pikselach.
 - *recoil_compensation(offset_x, offset_y)*: Wprowadza wirtualną poprawkę w osi pionowej symulującą przeciwdziałanie odrzutowi broni palnej, bazując na aktualnej broni.
 - *process_single_frame()*: Koordynuje pojedynczy cykl: wywołuje akwizycję obrazu, uruchamia inferencję sieci, aktualizuje stany wewnętrzne oraz opcjonalnie nakłada warstwę diagnostyczną OpenCV (okno debugowania).

- **Funkcja *vision_worker(pipe_conn, model_path, target_fps)***: Funkcja uruchomieniowa procesu Vision Worker. Kontroluje taktowanie pętli przetwarzania, nasłuchuje komend zmiany parametrów z kanału komunikacyjnego (np. zmiana priorytetu celu na TT/CT/ALL) i asynchronicznie przesyła wyliczone przesunięcia do procesu głównego.

Moduł akwizycji obrazu (*camera.py*) Warstwa abstrakcji sprzętowej izolująca platformę systemową od mechanizmu pobierania klatek obrazu.

- **Klasa *CameraProvider***: Zapewnia kompatybilność wieloplatformową potoku wizyjnego.
 - *__init__(region)*: Automatycznie wykrywa system operacyjny hosta. Na platformie Windows inicjalizuje bibliotekę *bettercam* wykorzystującą interfejs *NVIDIA Direct3D Desktop Duplication API*. Na platformach Linux uruchamia moduł *mss*.
 - *grab_gpu_tensor()*: Pobiera aktualny zrzut zdefiniowanego regionu ekranu i natychmiast mapuje go do struktury tensora PyTorch bezpośrednio w pamięci karty graficznej, całkowicie omijając pamięć RAM i narzut procesora.
 - *release()*: Zwalnia zasoby systemowe powiązane z przechwytywaniem obrazu.

Moduł komunikacji szeregowej i wejścia (*comms.py*) Odpowiada za warstwę transportową danych pomiędzy środowiskiem uruchomieniowym Python a mikrokontrolerem ESP32 oraz za przechwytywanie fizycznych naciśnięć klawiszy.

- **Klasa *SerialCommsModule***: Implementuje asynchroniczną komunikację UART poprzez port USB.
 - *connect()*: Otwiera port szeregowy z prędkością transmisji 115200 bodów i zarządza opóźnieniem wymaganym na restart układu po zestawieniu połączenia.
 - *check_connection(esp, port)*: Realizuje procedurę uwierzytelnienia. Wysyła zapytanie sekwencyjne "ESP32-CHECK" i oczekuje przez określony czas na flagę "ESP32-READY".
 - *send_command(command)*: Koduje tekstową ramkę sterującą do formatu UTF-8, dodaje znak powrotu karetki (*\r*) zgodny z protokołem i wymusza opróżnienie bufora wyjściowego.
 - *get_response()*: Odczytuje nadchodzące pakiety zwrotne z ESP32, dekoduje je i usuwa znaki niedrukowalne.
- **Klasa *KeyboardInputModule***: Niskopoziomowy wątek nasłuchujący zdarzeń klawiatury systemowej za pomocą biblioteki *pynput*.
 - *on_press(key) / on_release(key)*: Dodaje lub usuwa z wewnętrznej struktury zbioru zdefiniowane klawisze sterujące, eliminując problem powtórzeń systemowych.
 - *get_key()*: Przetwarza stan wciśniętych klawiszy. W pierwszej kolejności weryfikuje wystąpienie klawisza wyłączenia awaryjnego, a w przypadku jego braku generuje ciąg znaków reprezentujący aktualny wektor manualnego sterowania osiami.
- **Funkcja *comms_worker(conn)***: Funkcja zarządzająca procesem *Comms Worker*. Agreguje dane z potoku IPC interfejsu graficznego, wysyła sformatowane ramki sterujące (zawierające offsety, prędkość, stany przycisków myszy) z zachowaniem bezpiecznego interwału czasowego i przekazuje zwrotne logi diagnostyczne z ESP32 do interfejsu GUI.

Moduł filtrowania przycisków myszy (*mouse_blocker_worker.py*) Realizuje zadania związane z izolacją fizycznych peryferiów od działań systemu automatycznego sterowania.

- **Funkcja *mouse_blocker_worker(pipe_conn)***: W zależności od wykrytej platformy operacyjnej wywołuje dedykowane funkcje blokujące:
 - *_run_windows_blocker()*: Wstrzykuje filtr zdarzeń biblioteki *pynput* do kolejki komunikatów systemu Windows. Przechwytuje i blokuje kody komunikatów powiązane ze scrollmem, przyciskiem środkowym oraz przyciskami bocznymi, zapobiegając ich rejestracji przez system operacyjny.
 - *_run_linux_blocker()*: Przejmuje bezpośrednią kontrolę nad deskryptorem pliku fizycznej myszy w podsystemie *evdev* za pomocą metody *grab()*. Tworzy wirtualny interfejs wejściowy *UInput*, na który przekazuje wyłącznie surowe zdarzenia relatywne ruchu oraz podstawowe stany kliknięć (*BTN_LEFT*, *BTN_RIGHT*), całkowicie odcinając pozostałe przerwania sprzętowe.

4.3.3 Architektura interfejsu graficznego oraz inicjalizacja systemu

Zwieńczeniem architektury oprogramowania jest proces główny, który integruje wszystkie procesy pomocnicze i zapewnia interfejs graficzny. Został on zrealizowany przy użyciu biblioteki *Dear PyGui*.

Punkt wejścia i inicjalizacja (*app.py*) Skrypt startowy aplikacji, odpowiedzialny za bezpieczne powołanie do życia środowiska wieloprotocowego.

- Wykorzystuje bibliotekę *multiprocessing* do utworzenia niezależnych kanałów łączności dla każdego z workerów (wizyjnego, komunikacyjnego, symulacyjnego oraz blokady myszy).
- Odpowiada za zarządzanie cyklem życia procesów potomnych – ich poprawne uruchomienie, a w przypadku zamknięcia aplikacji (lub wystąpienia błędu krytycznego) – przesłanie sygnałów terminacji oraz zwolnienie zasobów za pomocą metody *join()*.
- Obsługuje dodatkowe flagi uruchomieniowe (np. *--export-only*), które pozwalają na wywołanie skryptu w trybie kompilacji przestrzennej modelu YOLO do zoptymalizowanego formatu TensorRT (silnik *.engine*) przed właściwym startem aplikacji.

Klasa interfejsu użytkownika (*gui.py*) Zarządza warstwą graficzną oraz dwukierunkową wymianą danych za pośrednictwem dedykowanych potoków komunikacyjnych.

- **Zarządzanie stanem i nawigacja:** Interfejs opiera się na wielozakładowym układzie z rozwijanym panelem bocznym. Przełączanie poszczególnych widoków (Strona Główna, Panel Sterowania, Symulacja, Statystyki, Ustawienia) realizowane jest poprzez manipulację właściwością widoczności odpowiednich grup widżetów.
- **Responsywność i skalowanie:** Aplikacja natywnie obsługuje rozdzielczości 720p oraz 1080p.
- **Wielojęzyczność:** Interfejs posiada wbudowany słownik, który mapuje etykiety tekstowe i opisy na wybrane warianty językowe (polski/angielski).
- **Wizualizacja telemetrii:** Aplikacja wykorzystuje wbudowane silniki renderowania wykresów do prezentacji danych w czasie rzeczywistym. Generowane są wykresy historii pozycji robota w osiach X i Y oraz dystansu do wykrytego celu.
- **Asynchroniczna pętla zdarzeń:** Aby uchronić główną pętlę renderującą Dear PyGui przed zablokowaniem, odbiór komunikatów od procesów pomocniczych został oddelegowany do osobnego wątku. Wątek ten w nieprzerwanej pętli odpytuje potoki za pomocą metody i w przypadku wykrycia nowych pakietów danych (np. statusu połączenia, logów UART z ESP32) aktualizuje odpowiednie zmienne stanu aplikacji oraz powiązane widżety tekstowe.

5 Napotkane problemy

5.1 Milestone I

- Zatkana dysza drukarki 3D
- Długi czas oczekiwania na zamówione części
- Brak pasujących sprzęgieł do enkoderów
- Wysoka cena komponentów

5.2 Milestone II

- Konieczność lekkiego redesignu UI/UX aplikacji w celu wprowadzenia nowych funkcjonalności
- Problemy z konwerterem poziomów logicznych
- Zepsute serwo
- Zrzucenie projektu na podłogę
- Wyginanie się elementów i luzy w konstrukcji chwytaka
- Święta - wyjazd członków zespołu
- Chwilowe zmniejszenie rozmiaru drużyny
- Dodrukowanie części z powodu złych pomiarów
- Przegrzewający się elektromagnes
- Zabrakło kabli

5.3 Milestone III

- Problemy z konwerterami, osią Z oraz solenoidami

- Spalony step-down
- Luźne sprzęgło
- Pęknięty kluczowy element robota oraz konieczność demontażu całej konstrukcji
- Spalony sterownik silnika NEMA23

6 Perspektywy rozwinięcia projektu

- Robot mógłby rysować przy zmianie końcówki.
- Robot mógłby grać w szachy przy zmianie końcówki.
- Robot mógłby służyć za niedokładną drukarkę 3D przy zmianie końcówki.
- Robot mógłby współpracować z systemem autonomicznego poruszania się po mapie CS. Taki system mógłby działać za pomocą AI i chodzić po mapie w losowe miejsca omijając przeszkody.

7 Załączniki

Załączniki znajdują się na repozytorium GitHub.

<https://github.com/ZSD-CSAimBot/CSAimBot>